

Lecture 8: struct your stuff

CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- Submit your github username via [form](#) on Piazza by tonight 11:59pm
 - Else, **cannot** receive HW2 !
- HW1 is due this Thursday at 11:59pm PT
- HW1 Quiz next Tuesday [in class](#)
 - 15min

What if I want to **insert** an element into an **array**?

- Ex: insert 6 at index 2

```
int arr[5] = {0, 1, 2, 3, 4};
```

int size = 5;

int arr[size] = { . . . };

~~size++;~~

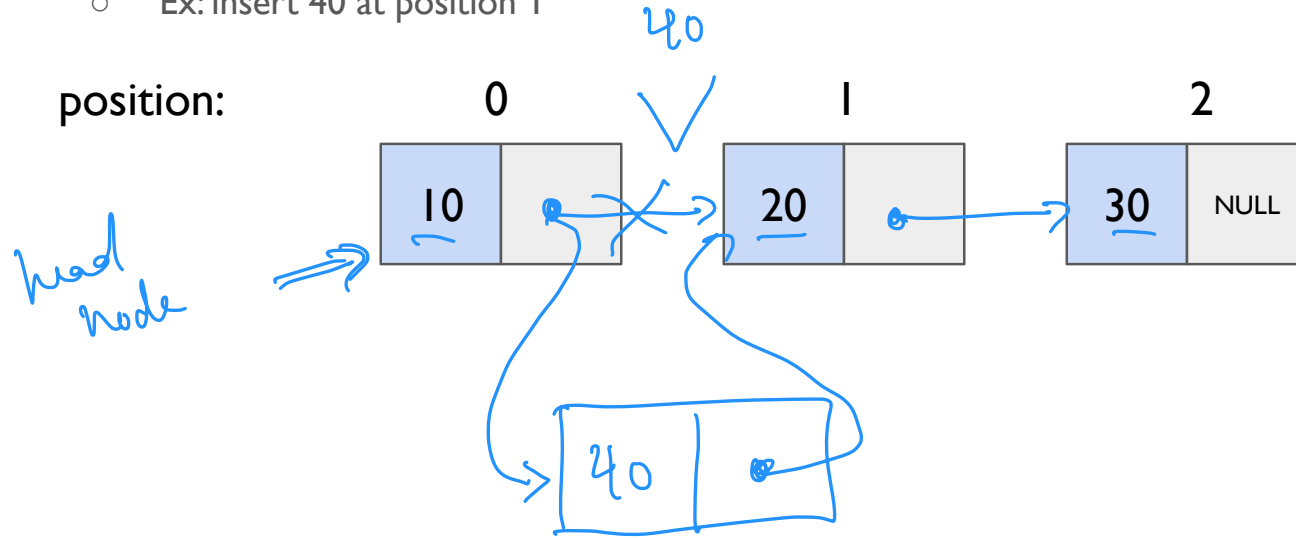
int arr2[size+1] = { 0, 1, 6, 2, 3, 4 };

$n = 1,000,000$

arr[100]; ✓

Linked List

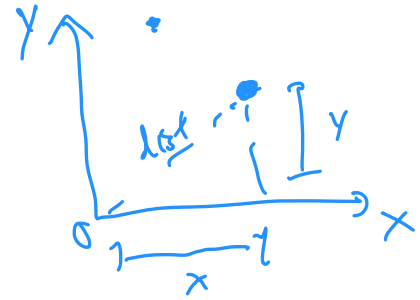
- A **dynamic** data structure: its **size** can grow and shrink
 - Ex: insert 40 at position 1



How can I represent Cartesian coordinates?

```
int x;  
int y;  
int z;  
int a;
```

(x, y)



class Point

```
int x;  
int y;
```

methods.

```
int pt[2];
```

```
pt[0] = 3; // x-coord
```

```
pt[1] = 4; // y-coord
```

Introducing struct datatype

- Up until now, all variables have been either single elements or arrays
 - Example: Cartesian coordinates

```
int x;
```

```
int y;
```

- The struct datatype can **combine** elements into **one variable**

```
struct point {
```

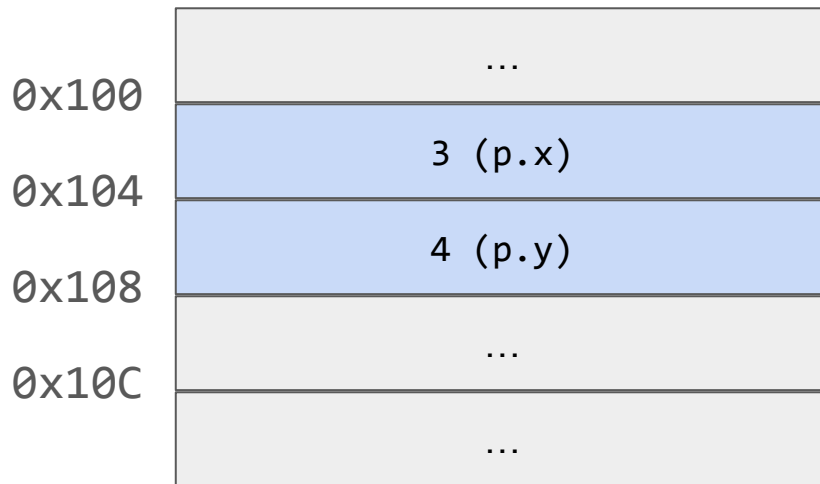
```
    int x; .
```

```
    int y; .
```

```
};
```

Using struct datatypes

```
struct point {  
    int x; // data member  
    int y;  
};  
  
int main() {  
    struct point p;  
  
    p.x = 3;  
    p.y = 4;  
    // or struct point p = {3, 4};  
}
```



Demo

```
struct point new_point(int x, int y);
```

Passing a struct to a function

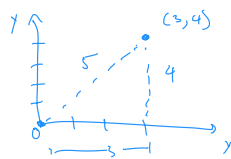
$$\underline{(x_1, y_1)} \quad \underline{(x_2, y_2)}$$

$$\text{dist} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

```
double distance(struct point p1, struct point p2) {
```

```
    // demo
```

```
}
```



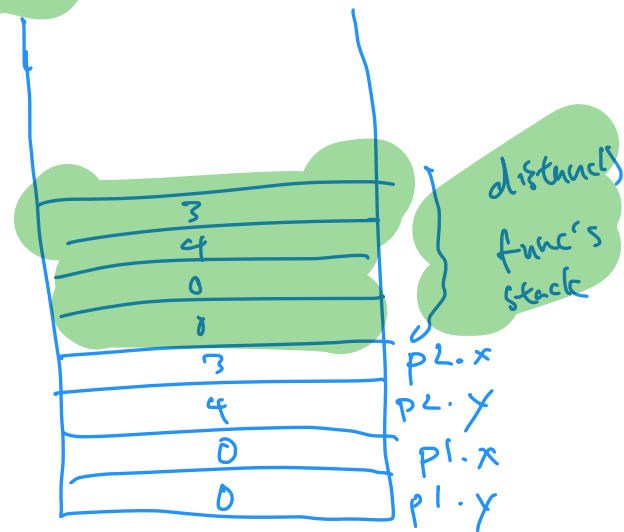
```
int main() {
```

```
    struct point p1 = new_point(0, 0);
```

```
    struct point p2 = new_point(3, 4);
```

```
    double dist = distance(p1, p2); // structs passed by value
```

```
}
```



structs are copied when passed as arguments to function

- Any issues with this?

`distance ( p1, p2 );`

wasteful for memory!

How can we avoid copying structs and wasting memory?

pointers!

How can we avoid **copying** structs?

- Pointers!

```
int main() {
```

```
    struct point p1;
```

struct ptr

```
    struct point *pp1 = &p1;
```

syntactic sugar

$(\&p1).x$

```
    pp1->x = 0;
```

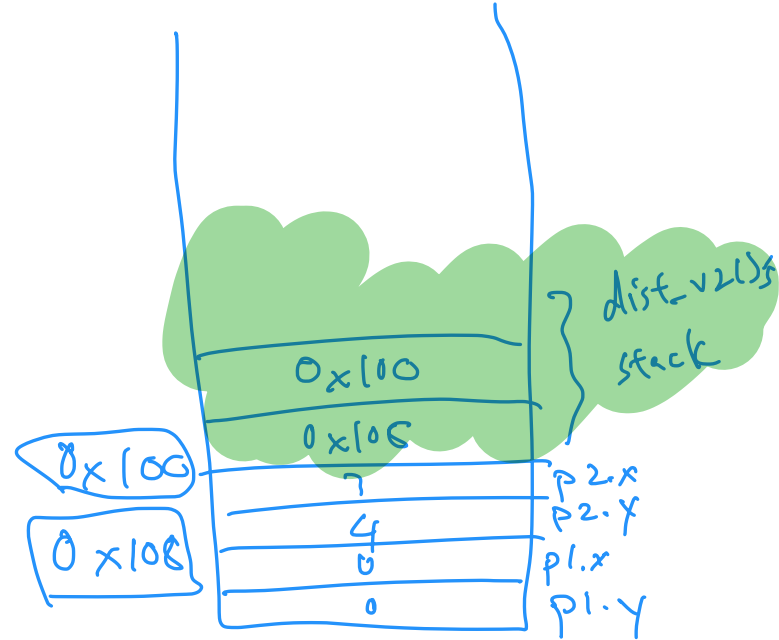
arrow operator

```
    pp1->y = 0;
```

```
}
```

Passing struct to a function using pointers

```
double distance_v2(struct point *p1, struct point *p2) {  
    // demo  
}
```

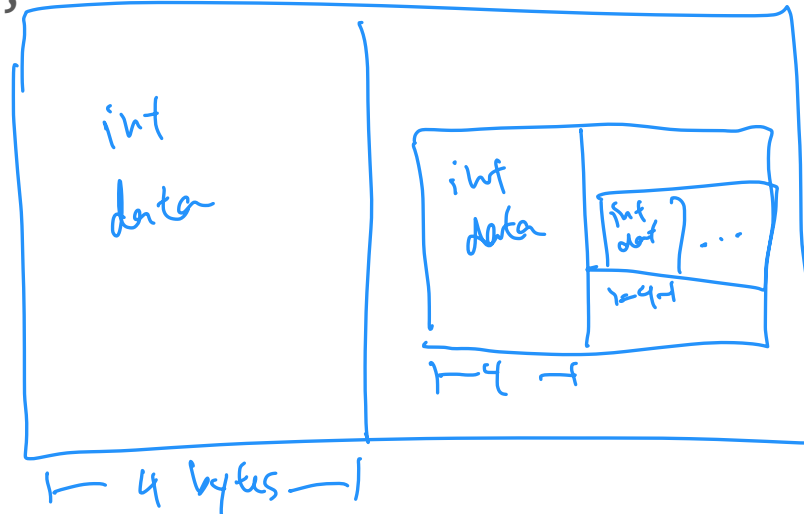


Linked list with struct

```
struct node {  
    int data;  
  
    struct node next;  
};
```

Is this kind of struct possible in C?

- A. Yes
- B. No



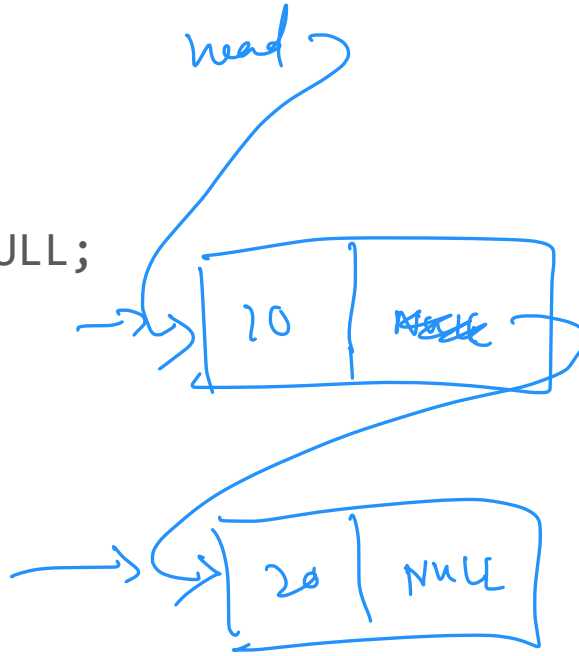
Linked list with struct

```
struct node {  
    int data; // 4 bytes  
    struct node *next; // pointer! // 8 bytes  
};
```

- Known as a self-referential struct

Linked List on the stack

```
int main() {  
    struct node *head = NULL;  
    struct node first;  
    first.data = 10;  
    first.next = NULL;  
  
    struct node second;  
    second.data = 20;  
    second.next = NULL;  
  
    first.next = &second;  
    head = &first;  
}
```



Find the bug!

```
1 void print_list(struct node *head) {  
2     struct node *current = head;  
3     while (current->next) {  
4         printf("|%d|%p| -> ", current->data, current->next);  
5         current = current->next;  
6     }  
7     printf("\n");  
8 }
```

- A. Line 2
- B. Line 3
- C. Line 4
- D. Line 5